ACTIVIDAD INDIVIDUAL: CONSUMO Y DOCUMENTACIÓN DE UNA API CON REACT

Componentes • Consumo de API • Fetch • Métodos HTTP • Manejo de respuestas • Pruebas • Git

Aprendiz: Cristian David Montilla Ordoñez

Programa / ficha: 3312932

Instructor(a): Yoly Alexandra Muñoz

Fecha de entrega: 24-octubre-2026

1. Contexto del reto

En la actividad anterior construiste una API utilizando Node.js y Express, con rutas HTTP, middleware, validaciones y respuestas en formato JSON.

Ahora debes desarrollar una interfaz web utilizando React que permita consumir los servicios de esa API. La interfaz deberá consultar información del servidor, enviar datos mediante formularios y mostrar al usuario las respuestas recibidas.

Este trabajo es individual. Debes investigar, diseñar, programar, probar y documentar tu solución. No se entrega código resuelto para copiar. Debes comprender y explicar las decisiones tomadas durante el desarrollo.

El frontend debe consumir la API desarrollada previamente, sin reemplazarla por datos simulados ni crear otro backend.

2. Objetivo de aprendizaje

Desarrollar una interfaz web con React que consuma una API creada con Express, utilizando fetch(), funciones asíncronas, eventos, componentes, estados y manejo de errores.

Además, debes documentar el proceso de construcción y explicar mediante un video el funcionamiento de la aplicación y la comunicación entre frontend y backend.

3. Requisitos funcionales

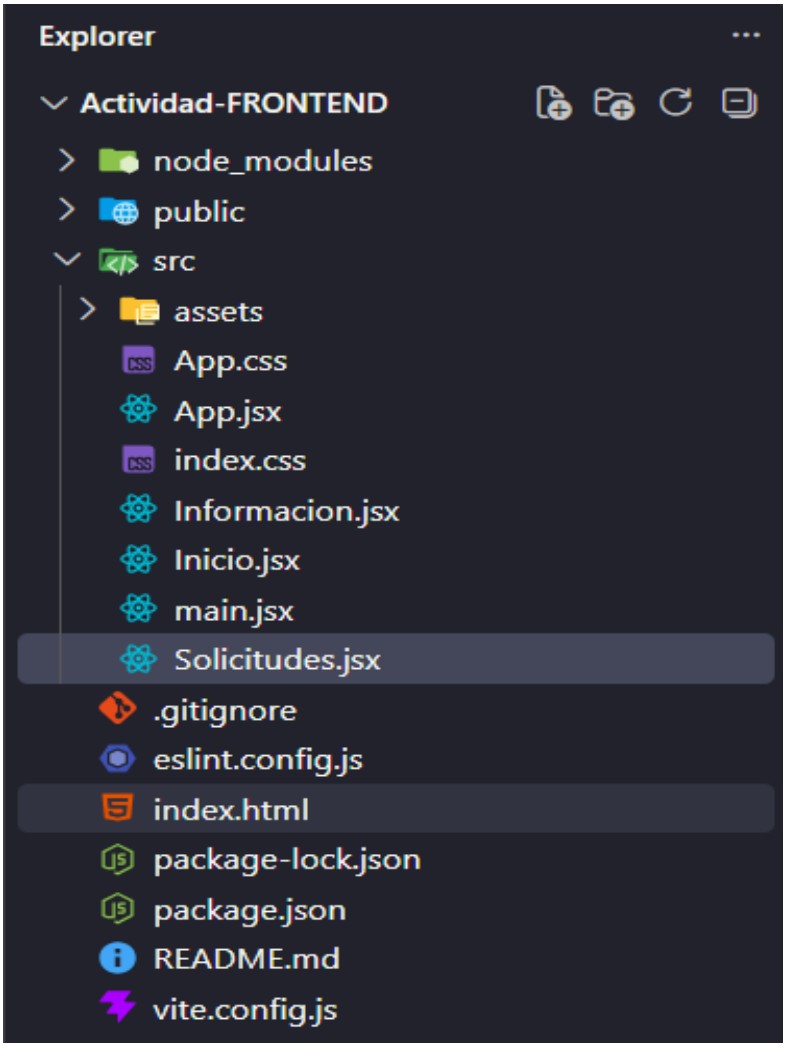
Debes construir una aplicación React que cumpla los siguientes requisitos:

3.1. Creación del proyecto

- Crear un proyecto React utilizando Vite.



- Organizar los archivos y componentes de manera clara.



- Identificar el archivo principal de la aplicación y el punto de entrada.
- Ejecutar el frontend en el entorno local.

- Verificar que la API de Express esté funcionando antes de realizar las pruebas.

3.2. Componente para consultar el endpoint GET /inicio

Crear un componente que permita:

- Consultar el endpoint GET /inicio mediante un botón.
- Mostrar en pantalla el mensaje recibido desde Express.
- Explicar cómo se realiza la petición HTTP.
- Explicar cómo se convierte la respuesta a JSON.
- Mostrar un mensaje comprensible si ocurre un error.

Solucion:

src > Inicio.jsx > Inicio

```jsx
import { useState } from "react"

function Inicio() {

    const [mensaje, setMensaje] = useState("")
    async function consumoApi() {

        try {
            let respuesta = await fetch("http://127.0.0.1:2000/inicio")
            let datos = await respuesta.json()

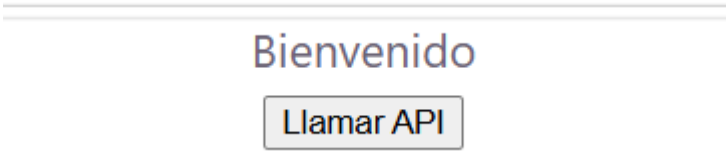
            if (respuesta.ok) {
                setMensaje(datos.mensaje)
            }

        } catch (error) {
            setMensaje("No se pudo conectar con Express.")
        }
    }

    return (
        <div>
            <h2>Inicio</h2>
            <button onClick={consumoApi}> Llamar API </button>
            <p>{mensaje}</p>
        </div>
    )
}

export default Inicio
```

```
app.get('/inicio', (req,res)=>{
    res.json({"mensaje":"Bienvenido"})
});
```



3.3. Componente para consultar el endpoint GET /informacion

Crear un componente independiente que permita:

- Consultar el endpoint GET /informacion mediante un botón.
- Mostrar la información que devuelve la API.
- Identificar las propiedades del objeto JSON recibido.
- Explicar cómo React utiliza los datos de la respuesta para actualizar la interfaz.

Solucion:

src > Informacion.jsx > Informacion > informacionAPI

```jsx
import { useState } from "react"

function Informacion() {

    const [servicios, setServicios] = useState("")
    const [version, setVersion] = useState("")
    const [descripcion, setDescripcion] = useState("")
    const [mensaje, setMensaje] = useState("")

    async function informacionAPI() {

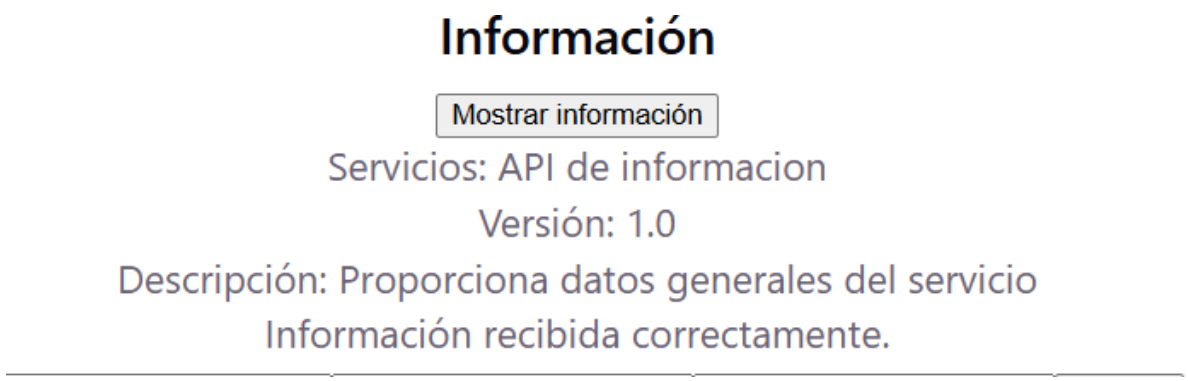
        try {
            let respuesta = await fetch("http://127.0.0.1:2000/informacion")
            let datos = await respuesta.json()

            if (respuesta.ok) {
                setServicios(datos.servicios)
                setVersion(datos.version)
                setDescripcion(datos.descripcion)
                setMensaje("Información recibida correctamente.")
            }

        } catch (error) {
            setMensaje("No se pudo conectar con Express.")
        }
    }

    return (
        <div>
            <h2>Información</h2>
            <button onClick={informacionAPI}> Mostrar información </button>
            <p>Servicios: {servicios}</p>
            <p>Versión: {version}</p>
            <p>Descripción: {descripcion}</p>
            <p>{mensaje}</p>
        </div>
    )
```

```
app.get('/informacion', (req,res)=>{
    res.json({
        "servicios":"API de informacion",
        "version":"1.0",
        "descripcion":"Proporciona datos generales del servicio"
    })
});
```



3.4. Componente para enviar solicitudes mediante POST /solicitudes

Crear un formulario que permita ingresar:

- Nombre.
- Apellido.
- Correo electrónico.

El formulario debe:

- Capturar los valores ingresados por el usuario.
- Evitar que la página se recargue al enviar la información.
- Enviar los datos al endpoint POST /solicitudes.
- Configurar correctamente el método HTTP y la cabecera Content-Type.
- Enviar el cuerpo de la petición en formato JSON.
- Mostrar la respuesta devuelta por Express.
- Informar si la API rechaza los datos por validación.
- Mostrar un mensaje cuando no sea posible establecer comunicación con el servidor.

No debes asumir que la solicitud fue exitosa únicamente porque se ejecutó fetch(). Debes comprobar el estado de la respuesta HTTP y presentar el resultado correspondiente.

Importante: utiliza exactamente los endpoints y los nombres de los campos definidos en tu backend. Si la respuesta de Express utiliza otros nombres de propiedades, adapta el frontend a ese contrato.

Solucion:

src > Solicitudes.jsx > Solicitudes > solicitudesApi

```jsx
import { useState } from "react"

function Solicitudes() {

    const [dato_nombre, setDatoNombre] = useState("")
    const [dato_apellido, setDatoApellido] = useState("")
    const [dato_correo, setDatoCorreo] = useState("")

    const [nombre, setNombre] = useState("")
    const [apellido, setApellido] = useState("")
    const [correo, setCorreo] = useState("")

    const [mensaje, setMensaje] = useState("")

    async function solicitudesApi() {

        if (nombre == "" || apellido == "" || correo == "") {
            setMensaje("Complete todos los campos")
        } else if (!correo.includes("@") || !correo.includes(".")) {
            setMensaje("Ingrese un correo válido")
        } else {
            try {
                let resultado = await fetch("http://127.0.0.1:2000/solicitudes", {
                    method: "POST",
                    body: JSON.stringify({
                        nombre: nombre,
                        apellido: apellido,
                        correo: correo
                    }),
                    headers: {
                        "Content-Type": "application/json"
                    }
                })

                let datos = await resultado.json()
```

```
36
37                  if (resultado.ok) {
38                      setDatoNombre(datos.dato_nombre)
39                      setDatoApellido(datos.dato_apellido)
40                      setDatoCorreo(datos.dato_correo)
41                      setMensaje(datos.mensaje)
42                  }
43
44              } catch (error) {
45                  setMensaje("No se pudo conectar con Express")
46              }
47          }
48      }
49
50      return (
51          <div>
52              <input placeholder="Ingrese el nombre" onChange={(e) => setNombre(e.target.value)}/>
53              <input placeholder="Ingrese el apellido" onChange={(e) => setApellido(e.target.value)}/>
54              <input type="email" placeholder="Ingrese el correo" onChange={(e) => setCorreo(e.target.value)}/>
55              <button onClick={solicitudesApi}>Aceptar</button>
56              <p>{dato_nombre}</p>
57              <p>{dato_apellido}</p>
58              <p>{dato_correo}</p>
59              <p>{mensaje}</p>
60          </div>
61      )
62  }
63
64  export default Solicitudes
```

```
35
36  app.post('/solicitudes', (req,res)=>{
37      const {nombre,apellido,correo}=req.body
38      res.json({
39          "dato_nombre": nombre,
40          "dato_apellido": apellido,
41          "dato_correo": correo
42      })
43  })
```

Cristian | Montilla | montilla@gmail.com | Aceptar

Cristian

Montilla

montilla@gmail.com

Headers | Payload | Preview | Response | Initiator | Timing

▼ {dato_nombre: "Cristian", dato_apellido: "Montilla", dato_correo: "montilla@gmail.com"}
  dato_apellido: "Montilla"
  dato_correo: "montilla@gmail.com"
  dato_nombre: "Cristian"

4. Requisitos técnicos del frontend

La aplicación debe cumplir con las siguientes condiciones:

- Utilizar React y JavaScript.
- Crear componentes independientes para las consultas y el formulario.
- Utilizar useState para gestionar los datos ingresados y las respuestas que deban mostrarse.
- Utilizar funciones ejecutadas mediante botones o el envío del formulario.
- Utilizar fetch() para consumir la API.
- Utilizar async/await para gestionar las operaciones asíncronas.
- Convertir las respuestas JSON utilizando response.json().
- Presentar mensajes de carga, éxito y error cuando corresponda.
- No utilizar useEffect para realizar las consultas de esta actividad: las peticiones deben ejecutarse por interacción del usuario.
- No instalar ni utilizar Axios; el propósito es practicar el consumo de API mediante fetch().
- No modificar el backend para solucionar problemas que deban resolverse desde el frontend, salvo que se identifique una incompatibilidad real entre ambos proyectos y se documente.

5. Investigación obligatoria sobre consumo de API

Investiga y responde con tus propias palabras las siguientes preguntas. Incluye las fuentes consultadas.

1. **¿Qué es una API y para qué sirve en una aplicación web?**

Una API es una forma de comunicación entre dos programas. Sirve para que una aplicación pueda pedir o enviar información a otra. Por ejemplo, React se comunica con Express para solicitar información o enviar los datos de un formulario.

2. **¿Qué diferencia existe entre frontend y backend?**

El frontend es la parte de la aplicación que ve y utiliza el usuario, como los botones, formularios y textos. El backend es la parte que trabaja detrás de la aplicación, recibe las

solicitudes, procesa los datos y devuelve respuestas. En mi proyecto, React es el frontend y Express es el backend.

3. **¿Qué significa consumir una API?**

Consumir una API significa utilizarla para pedir o enviar información. Por ejemplo, cuando presiono un botón en React para consultar el endpoint /inicio, estoy consumiendo la API de Express.

4. **¿Qué función cumple fetch()?**

fetch() sirve para realizar peticiones a una dirección. Con ella puedo consultar información de Express o enviar datos desde React. También permite recibir la respuesta del servidor.

5. **¿Por qué se utilizan async y await en una petición HTTP?**

Se utilizan para trabajar más fácil con operaciones que pueden tardar en responder, como una petición a una API. async permite crear una función asíncrona y await permite esperar la respuesta antes de continuar con la siguiente instrucción.

6. **¿Qué es una promesa en JavaScript?**

Una promesa es un objeto que representa el resultado de una operación que todavía no ha terminado. Puede completarse correctamente o presentar un error. Por ejemplo, cuando utilizo fetch(), obtengo una promesa que representa la petición al servidor.

7. **¿Para qué sirve response.json()?**

Sirve para leer los datos de la respuesta y convertir el contenido JSON en un objeto de JavaScript. De esta manera puedo acceder a sus propiedades y mostrar la información en React.

8. **¿Qué diferencia existe entre una petición GET y una petición POST?**

GET se utiliza principalmente para consultar información del servidor. POST se utiliza para enviar datos al servidor. En mi proyecto uso GET para consultar /inicio y /informacion, y POST para enviar los datos a /solicitudes.

9. **¿Qué función cumplen las cabeceras HTTP y por qué se utiliza Content-Type: application/json?**

Las cabeceras HTTP permiten enviar información adicional sobre una petición o una respuesta. Content-Type: application/json indica que los datos que estoy enviando en el cuerpo de la petición están en formato JSON, para que el servidor sepa cómo interpretarlos.

10. ¿Por qué se utiliza JSON.stringify() al enviar datos en una petición POST?

Se utiliza para convertir un objeto de JavaScript en un texto con formato JSON. Así puedo enviar los datos del formulario, como el nombre, el apellido y el correo, para que Express los reciba y los procese.

11. ¿Qué diferencia existe entre un error de conexión y una respuesta HTTP con código 400?

Un error de conexión ocurre cuando no se logra establecer la comunicación o no se puede acceder a la respuesta, por ejemplo, si el servidor está apagado. En cambio, una respuesta HTTP 400 significa que el servidor sí respondió, pero considera que la solicitud tiene un problema, como datos incorrectos o una petición mal hecha.

12. ¿Qué función cumplen useState y los eventos onClick y onSubmit?

useState sirve para guardar información dentro de un componente y actualizarla cuando cambia. onClick ejecuta una función cuando se presiona un botón, mientras que onSubmit ejecuta una función cuando se envía un formulario. Si se utiliza onSubmit, se puede usar preventDefault() para evitar que la página se recargue.

13. ¿Qué es CORS y por qué puede afectar la comunicación entre React y Express?

CORS es una regla de seguridad que controla si una página puede acceder a recursos de otro origen. Puede afectar la comunicación porque React y Express pueden estar funcionando en puertos diferentes. Si Express no permite el origen desde el que se ejecuta React, el navegador puede bloquear el acceso a la respuesta.

14. ¿Qué ocurre si el frontend envía propiedades diferentes a las que espera el backend?

Puede ocurrir que Express no encuentre los datos que necesita y la solicitud no funcione correctamente. Por ejemplo, si el backend espera una propiedad llamada nombre y React envía nombreUsuario, el backend no recibirá el dato bajo el nombre que espera. Por eso es importante utilizar los mismos nombres de propiedades en ambos lados.

15. ¿Por qué no se debe considerar exitosa toda petición solamente porque fetch() devolvió una respuesta?

Porque recibir una respuesta no significa que la operación haya salido bien. El servidor puede responder con códigos como 400 o 500. Por eso se debe comprobar el estado HTTP, por ejemplo, con respuesta.ok, para saber si la petición fue exitosa.

Fuentes consultadas

1. MDN Web Docs. Concepto de API: https://developer.mozilla.org/en-US/docs/Glossary/API

2. MDN Web Docs. Uso de Fetch: https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

3. MDN Web Docs. Promesas en JavaScript: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Using_promises

4. MDN Web Docs. Funciones async: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function

5. React. Estado de los componentes: https://react.dev/learn/state-a-components-memory

6.React. Eventos en los componentes: https://react.dev/learn/responding-to-events

7. MDN Web Docs. Métodos GET y POST: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods

8. MDN Web Docs. Cabecera Content-Type: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/Content-Type

9. MDN Web Docs. JSON.stringify(): https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

10. MDN Web Docs. CORS: https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CORS

11. MDN Web Docs. Código de estado HTTP 400: https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status/400

6. Pruebas obligatorias

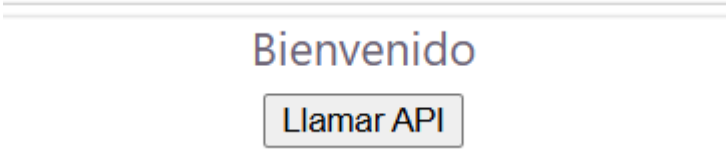
Realiza las siguientes pruebas y adjunta capturas de pantalla propias.

Prueba	Resultado esperado

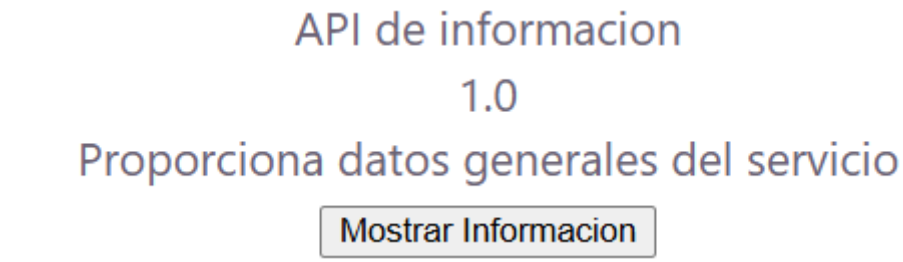
Consultar /inicio	Mostrar el mensaje recibido de Express.
Consultar /informacion	Mostrar la información devuelta por la API.
Enviar un formulario completo	Mostrar la respuesta de la API.
Enviar campos vacíos	Mostrar las validaciones que correspondan.
Enviar un correo inválido	Mostrar el rechazo de la API o la validación implementada en el frontend.
Apagar temporalmente Express y realizar una consulta	Mostrar un mensaje de error de conexión.
Enviar una solicitud con datos inválidos	Mostrar el mensaje de error devuelto por el backend.

Solución:

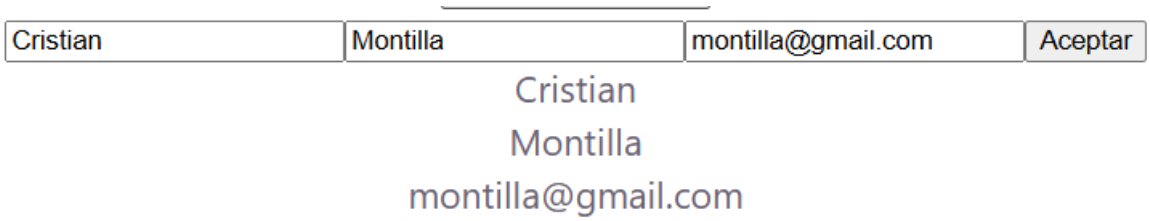
1.



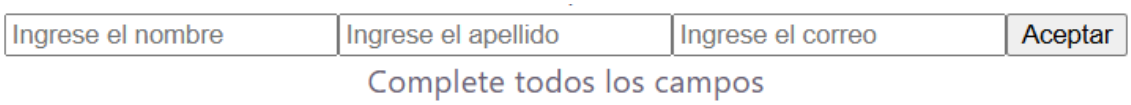
2.



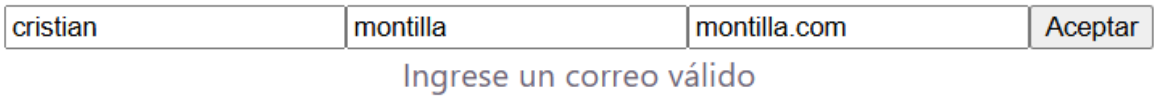
3.



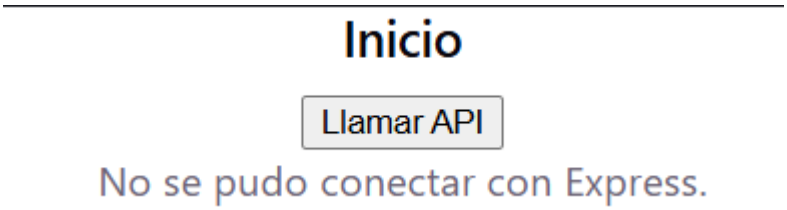
4.



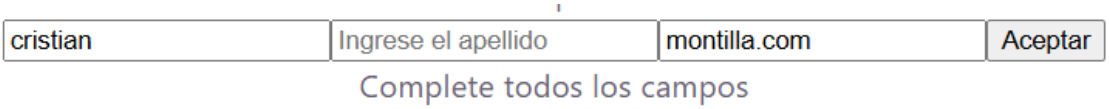
5.



6.



7.



En las evidencias debes demostrar el comportamiento de la interfaz y, cuando corresponda, mostrar en las herramientas de desarrollador del navegador la petición realizada, el método HTTP, el código de estado y la respuesta recibida.

No debes inventar resultados ni presentar capturas de otra persona.

7. Documento técnico obligatorio

Elabora un documento de entre 4 y 7 páginas, sin contar la portada ni los anexos, en el que expliques el proceso completo de desarrollo del frontend.

El documento debe incluir:

Portada

- Nombre de la actividad.

- Nombre del aprendiz.
- Programa y ficha.
- Nombre del instructor.
- Fecha de entrega.

Introducción y objetivo

Explica qué desarrollaste, qué necesidad resuelve y cuál es la relación entre React y Express.

Instalación y configuración

Explica:

- Para qué sirven Node.js y npm.
- Cómo creaste el proyecto con Vite.
- Qué función cumple cada archivo importante.
- Cómo ejecutas el frontend.
- Cómo verificaste que el backend estuviera funcionando.

Desarrollo de los componentes

Explica individualmente los componentes Inicio, Informacion y Solicitudes.

Para cada uno debes indicar:

- Qué responsabilidad cumple.
- Qué endpoint consume.
- Qué método HTTP utiliza.
- Cómo se ejecuta la petición.
- Cómo recibe y procesa la respuesta.
- Cómo muestra la información.
- Qué errores pueden presentarse y cómo los manejaste.

Explicación del código

Incluye fragmentos de tu propio código y explica qué hace cada parte relevante. No es suficiente pegar el código sin justificar su funcionamiento.

Debes explicar especialmente fetch(), async/await, response.json(), JSON.stringify(), useState, los eventos y el manejo de errores.

8.6. Pruebas y resultados

Incluye las capturas de las pruebas realizadas y describe el resultado obtenido en cada caso.

8.7. Dificultades y soluciones

Describe al menos dos dificultades reales que hayas encontrado durante el desarrollo. Explica cuál era el problema, cómo lo identificaste, qué solución aplicaste y cómo verificaste que funcionara.

Si no encontraste dos errores, puedes realizar pruebas controladas para estudiar problemas como una URL incorrecta, el servidor apagado o una respuesta HTTP de error.

8.8. Conclusiones y referencias

Redacta conclusiones personales sobre lo aprendido y agrega las referencias consultadas con sus enlaces.

9. Video de sustentación individual

Graba un video en el que expliques y demuestres el funcionamiento de tu aplicación.

Duración sugerida: entre 5 y 8 minutos.

El video debe incluir:

1. **Presentación:** identifica la actividad y explica brevemente su objetivo.
2. **Estructura:** muestra los archivos y componentes que desarrollaste.
3. **Consulta de información:** ejecuta los botones de /inicio y /informacion.
4. **Envío de datos:** diligencia el formulario y demuestra cómo React envía la petición POST.
5. **Respuesta del backend:** muestra los datos recibidos y explica cómo se presentan en pantalla.
6. **Manejo de errores:** demuestra al menos una prueba fallida y explica el mensaje que presenta la aplicación.
7. **Herramientas del navegador:** muestra una petición en la pestaña Network e identifica el método, el estado HTTP y la respuesta.

8. **Explicación técnica:** describe cómo se relacionan React, fetch() y Express.
9. **Conclusión:** explica qué aprendiste y qué mejorarías en una próxima versión.

El video debe permitir observar la aplicación y escuchar tu explicación. No basta con leer el documento o mostrar una presentación. Debes demostrar que comprendes el código que desarrollaste.